

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.(BL3)*

Assessment Tool Weightage:20%

QUESTIONS: 50

**Duration :60 Mins
On Class
Total Marks:50**

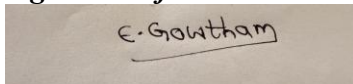
Annexure A

Viva Voce

Code of Conduct:

*I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in black ink that reads "E. Gowtham".

Annexure A

Viva Voce

- Q1. What is code optimization?
- Q2. Why is optimization important in a compiler?
- Q3. What are the goals of code optimization?
- Q4. What are the principal sources of optimization?
- Q5. Give an example of redundant computation.
- Q6. What is constant folding?
- Q7. What is constant propagation?
- Q8. What is strength reduction?
- Q9. What is algebraic simplification?
- Q10. What is loop optimization?
- Q11. What is peephole optimization?
- Q12. Why is it called "peephole"?
- Q13. What are the types of peephole optimizations?
- Q14. How does peephole optimization remove redundant instructions?
- Q15. What is null sequence elimination?
- Q16. What is instruction replacement?
- Q17. Give an example of algebraic simplification in peephole optimization.
- Q18. What is redundant load/store elimination?
- Q19. What is constant folding in peephole optimization?
- Q20. What is strength reduction in peephole optimization?
- Q21. What is a basic block?
- Q22. What are the properties of a basic block?
- Q23. What is a leader statement?
- Q24. What are the rules to identify leaders?
- Q25. What is a control flow graph (CFG)?
- Q26. What are nodes and edges in CFG?
- Q27. How are basic blocks connected in a CFG?
- Q28. Why are basic blocks important for optimization?

- Q29. What is a Directed Acyclic Graph (DAG)?
- Q30. Why is DAG used in basic block optimization?
- Q31. How is a DAG constructed?
- Q32. What is a leaf node in DAG?
- Q33. What is an interior node in DAG?
- Q34. How does DAG help in common subexpression elimination?
- Q35. How does DAG help in dead code elimination?
- Q36. What is a label in DAG nodes?
- Q37. What is the difference between DAG and syntax tree?
- Q38. What are the advantages of DAG representation?
- Q39. What is live variable analysis?
- Q40. What is available expression analysis?
- Q41. What is the difference between local and global optimization?
- Q42. What is data dependency?
- Q43. What is a code generator?
- Q44. What are the issues in code generator design?
- Q45. What is a target machine?
- Q46. What is instruction selection?
- Q47. What are addressing modes?
- Q48. What is register allocation?
- Q49. What is register spilling?
- Q50. What is next-use information, and why is it important?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively

Code Optimization

Q1. What is code optimization?

Code optimization is the process of improving code so that it executes **faster and/or uses fewer resources** without changing its output.

Q2. Why is optimization important in a compiler?

It improves **execution speed, memory usage, and overall performance** of the generated program.

Q3. What are the goals of code optimization?

The main goals are:

- Reduce execution time
- Reduce memory usage
- Reduce code size
- Improve resource utilization
- Maintain the same program output

Q4. What are the principal sources of optimization?

The main sources are:

- Redundant computations
- Constant expressions
- Common subexpressions
- Dead code
- Loops
- Inefficient instructions

Q5. Give an example of redundant computation.

Example:

$a = b + c$

$d = b + c$

The expression $b + c$ is calculated twice. It can be calculated once and reused.

Q6. What is constant folding?

Constant folding means **evaluating constant expressions at compile time**.

Example:

$x = 10 * 5$

becomes:

$x = 50$

Q7. What is constant propagation?

Constant propagation means **replacing a variable with its known constant value**.

Example:

$x = 10$

$y = x + 5$

becomes:

$y = 10 + 5$

Q8. What is strength reduction?

Strength reduction replaces an **expensive operation with a cheaper operation**.

Example:

$x = y * 2$

can become:

$x = y + y$

Q9. What is algebraic simplification?

It simplifies expressions using algebraic rules.

Examples:

$x + 0 \rightarrow x$

$x * 1 \rightarrow x$

$x * 0 \rightarrow 0$

Q10. What is loop optimization?

Loop optimization improves loops to **reduce the number of operations and execution time**. Examples include loop-invariant code motion, loop unrolling, and strength reduction.

Peephole Optimization

Q11. What is peephole optimization?

Peephole optimization examines a **small sequence of consecutive instructions** and replaces it with a more efficient sequence.

Q12. Why is it called "peephole"?

It is called peephole because the optimizer looks through a **small "window" or peephole** at a few instructions at a time.

Q13. What are the types of peephole optimizations?

Common types are:

1. Redundant instruction elimination
2. Null sequence elimination
3. Instruction replacement
4. Algebraic simplification
5. Strength reduction
6. Redundant load/store elimination
7. Constant folding

Q14. How does peephole optimization remove redundant instructions?

It identifies instructions whose results are **not needed or are already available** and removes them.

Example:

MOV R1, R2

MOV R1, R2

The second instruction is unnecessary.

Q15. What is null sequence elimination?

It removes instructions that **have no useful effect**.

Example:

ADD R1, 0

can simply be removed.

Q16. What is instruction replacement?

It replaces an instruction with an **equivalent but more efficient instruction**.

Example:

MUL R1, 2

can be replaced with a shift instruction when supported by the target machine.

Q17. Give an example of algebraic simplification in peephole optimization.

ADD R1, 0

can be replaced by nothing because adding zero does not change the value.

Q18. What is redundant load/store elimination?

It removes unnecessary **LOAD or STORE instructions** when the required value is already available in a register or memory.

Q19. What is constant folding in peephole optimization?

It evaluates constant operations directly.

Example:

MOV R1, 5

ADD R1, 3

can be replaced with:

MOV R1, 8

Q20. What is strength reduction in peephole optimization?

It replaces an expensive instruction with a cheaper equivalent instruction.

Example:

MUL R1, 2

→

SHL R1, 1

Basic Blocks and CFG

Q21. What is a basic block?

A basic block is a **sequence of consecutive instructions with one entry point and one exit point**.

Q22. What are the properties of a basic block?

- It has a single entry point.
- Control enters at the beginning.
- It has a single exit point.
- There is no branching inside except at the end.

Q23. What is a leader statement?

A leader is the **first statement of a basic block**.

Q24. What are the rules to identify leaders?

A statement is a leader if:

1. It is the **first statement** of the program.
2. It is the **target of a jump**.
3. It immediately follows a **conditional or unconditional jump**.

Q25. What is a Control Flow Graph (CFG)?

A CFG is a graph that represents the **flow of control between basic blocks**.

Q26. What are nodes and edges in CFG?

- **Nodes:** Basic blocks.
- **Edges:** Possible flow of control between basic blocks.

Q27. How are basic blocks connected in a CFG?

Basic blocks are connected using directed edges representing possible **execution paths**.

Q28. Why are basic blocks important for optimization?

They provide a convenient unit for analyzing and optimizing code, especially for **local optimization and control-flow analysis**.

DAG

Q29. What is a Directed Acyclic Graph (DAG)?

A DAG is a directed graph with **no cycles**. In compiler optimization, it represents computations within a basic block.

Q30. Why is DAG used in basic block optimization?

DAG helps identify:

- Common subexpressions
- Redundant computations
- Dead code
- Dependencies between expressions

Q31. How is a DAG constructed?

A DAG is constructed by:

1. Creating leaf nodes for input variables/constants.
2. Creating interior nodes for operators.

3. Connecting operands to operators.
4. Reusing an existing node when the same expression occurs again.

Q32. What is a leaf node in DAG?

A leaf node represents an **input value, variable, or constant**.

Q33. What is an interior node in DAG?

An interior node represents an **operation or expression**, such as +, -, *, or /.

Q34. How does DAG help in common subexpression elimination?

If the same expression occurs more than once, DAG represents it using **one common node**, so the computation can be performed only once.

Example:

a = b + c

d = b + c

Both can share the same b + c node.

Q35. How does DAG help in dead code elimination?

DAG identifies computations whose results are **never used**, allowing those computations to be removed.

Q36. What is a label in DAG nodes?

A label identifies the **variable or value associated with a DAG node**.

Q37. What is the difference between DAG and syntax tree?

Syntax Tree	DAG
May duplicate common expressions	Shares common expressions
Represents expression structure	Represents expression structure and sharing
Used mainly for syntax representation	Used mainly for optimization
Can contain repeated subtrees	Avoids repeated computation

Q38. What are the advantages of DAG representation?

- Detects common subexpressions
- Helps eliminate redundant computations
- Helps identify dead code
- Shows dependencies
- Improves basic block optimization

Data-Flow Analysis

Q39. What is live variable analysis?

Live variable analysis determines whether a variable's **current value may be used in the future**.

If a value will never be used again, its computation can potentially be removed.

Q40. What is available expression analysis?

It determines which expressions have **already been computed and are still valid** at a particular point in the program.

It helps perform common subexpression elimination.

Q41. What is the difference between local and global optimization?

Local Optimization	Global Optimization
Works within one basic block	Works across multiple basic blocks
Simpler	More complex
Limited scope	Wider scope
Example: constant folding	Example: global common subexpression elimination

Q42. What is data dependency?

Data dependency occurs when one instruction **depends on the value produced or modified by another instruction**.

Example:

$a = b + c$

$d = a + 5$

The second statement depends on the first because it uses a .

Code Generation**Q43. What is a code generator?**

A code generator is the compiler phase that converts **intermediate code into target machine code or assembly code**.

Q44. What are the issues in code generator design?

Important issues include:

- Instruction selection
- Register allocation
- Register assignment
- Addressing modes
- Evaluation order
- Memory management
- Code quality

Q45. What is a target machine?

A target machine is the **hardware architecture for which the compiler generates code**.

Examples: x86, ARM, RISC-V.

Q46. What is instruction selection?

Instruction selection is choosing the **most suitable target-machine instructions** to implement intermediate code.

Q47. What are addressing modes?

Addressing modes specify **how an instruction accesses its operands**.

Examples:

- Immediate addressing
- Register addressing
- Direct addressing
- Indirect addressing
- Indexed addressing

Q48. What is register allocation?

Register allocation assigns frequently used variables and temporary values to **CPU registers** to improve execution speed.

Q49. What is register spilling?

Register spilling occurs when there are **more values to store than available registers**. Some values are temporarily stored in memory.

Q50. What is next-use information, and why is it important?

Next-use information tells **when a variable will be used again in the future**.

It helps the compiler decide:

- Which value should remain in a register.
- Which register can be reused.
- Which value should be spilled to memory.